交易订单表如何做索引优化

原创 营销&交易技术 大淘宝技术 2025年11月19日 01:15 浙江 509人

本文首先以淘天电商交易订单表线上一条非典型慢 SQL 的深入剖析为切入点，示范如何系统地分析与排查慢 SQL；接着详尽归纳了索引分类、B+Tree 与 B-Tree 的结构差异、B+Tree 高度估算方法、EXPLAIN 与 Query Profile 等诊断工具的使用，以及索引下推与排序的执行流程等索引优化理论；最后结合日常实践经验，提出了适用于大规模线上集群的索引变更 SOP，并总结了常见的慢 SQL 成因与相应的解决策略。

01 前言

交易订单表（tcorder）用于存储集团电商的在线订单记录，该表近60个字段，单个分表近千万行左右（受历史订单迁移影响会上下浮动），平均行长5.4KB，是名副其实的大表，该表的读写性能直接影响上游创单、逆向退款、订单列表等一系列跟订单有关的业务功能，对该表的任何变更都是非常谨慎，需要多方一起综合评估。受业务需求推动，近两年我非常“有幸”各操作了一次订单表索引优化，这里总结下mysql索引优化相关的知识、SOP、之前遇到的慢SQL问题及其对应的优化方法。

02 一个非典型的慢SQL

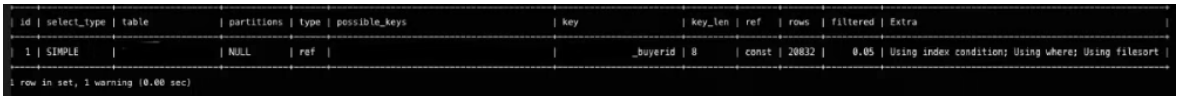
7月份做订单表的慢SQL梳理，发现分页查询类的请求比较多，典型的SQL如下：

```
select order_id from tcorder where is_main = 1
  and buyer_id=2********5
  ***************
  order by create_time desc,order_id asc limit 0,10
```

该类慢SQL执行的统计信息如下：

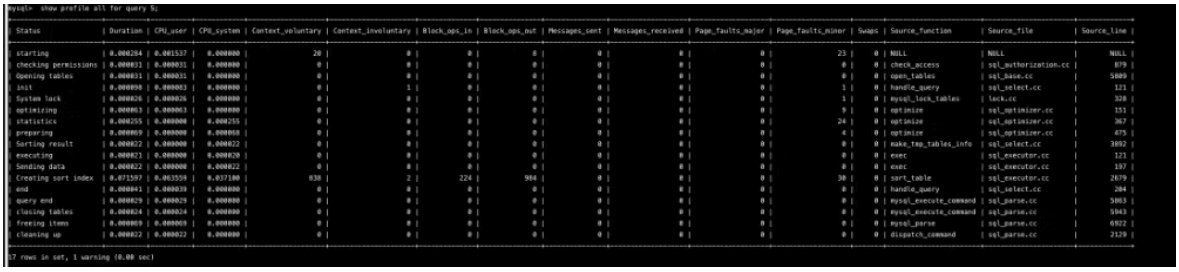
字段	结果	说明
sql_id	71d14de3	SQL Parttern ID，基于原始SQL文本通过BKDR的HASH算法生成
execute_count	3106	慢SQL总次数
cost	2017	平均执行耗时，单位ms
send_row_count	10	平均返回行数
examined_row_count	8705	平均扫描行数
logical_read	40023	平均逻辑读，即SQL处理过程中读取的数据页和索引页的数量，包含从Innodb Buffer Pool和磁盘读取两部分的，数据页和索引页的大小都是16kb
physical_sync_read	5174	平均同步物理读，即SQL处理过程中从磁盘读取的数据页和索引页的数量

该慢SQL的执行计划如下：

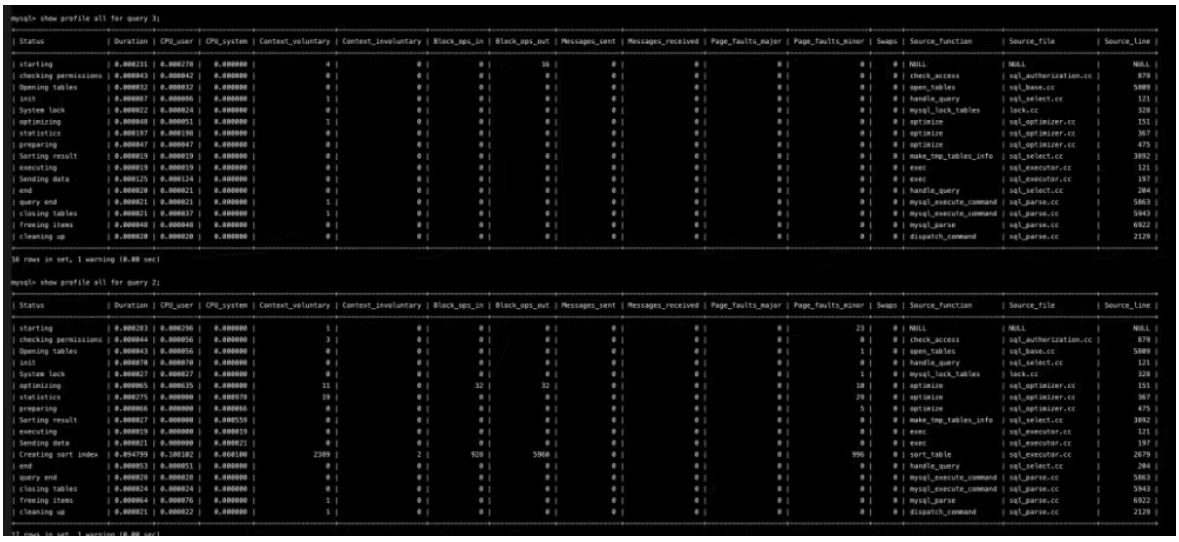


据此该SQL实际执行过程中是命中索引 `ind_***_buyerid` ，其中buyer_id走索引过滤（key_len等于8），is_main等大部分字段都在索引里面，通过索引下推过滤（Using index conditions），只有两个字段不在索引中通过回表查过滤（Using where）。这两个字段正常满足条件的比例不超过1%，因此即使回表查扫描行数也不会大幅上涨。出现问题的可能原因就是这里的Using filesort，这种方式下需要把所有满足条件的记录都过滤出来再排序，排序完成再取前10条，导致扫描行数和逻辑读异常。

上述猜想可以通过mysql的show profile执行过程来验证，如下：



Creating sort index的耗时和CPU损耗远大于其他几步，确认是filesort导致。为啥出现filesort呢？因为排序条件create_time desc,order_id asc无法利用索引。create_time在索引ind_***_buyerid里面，order_id是主键索引，但是这两个是不同的索引，多字段排序时无法利用索引排序。同样可以通过show profile来验证，如下：

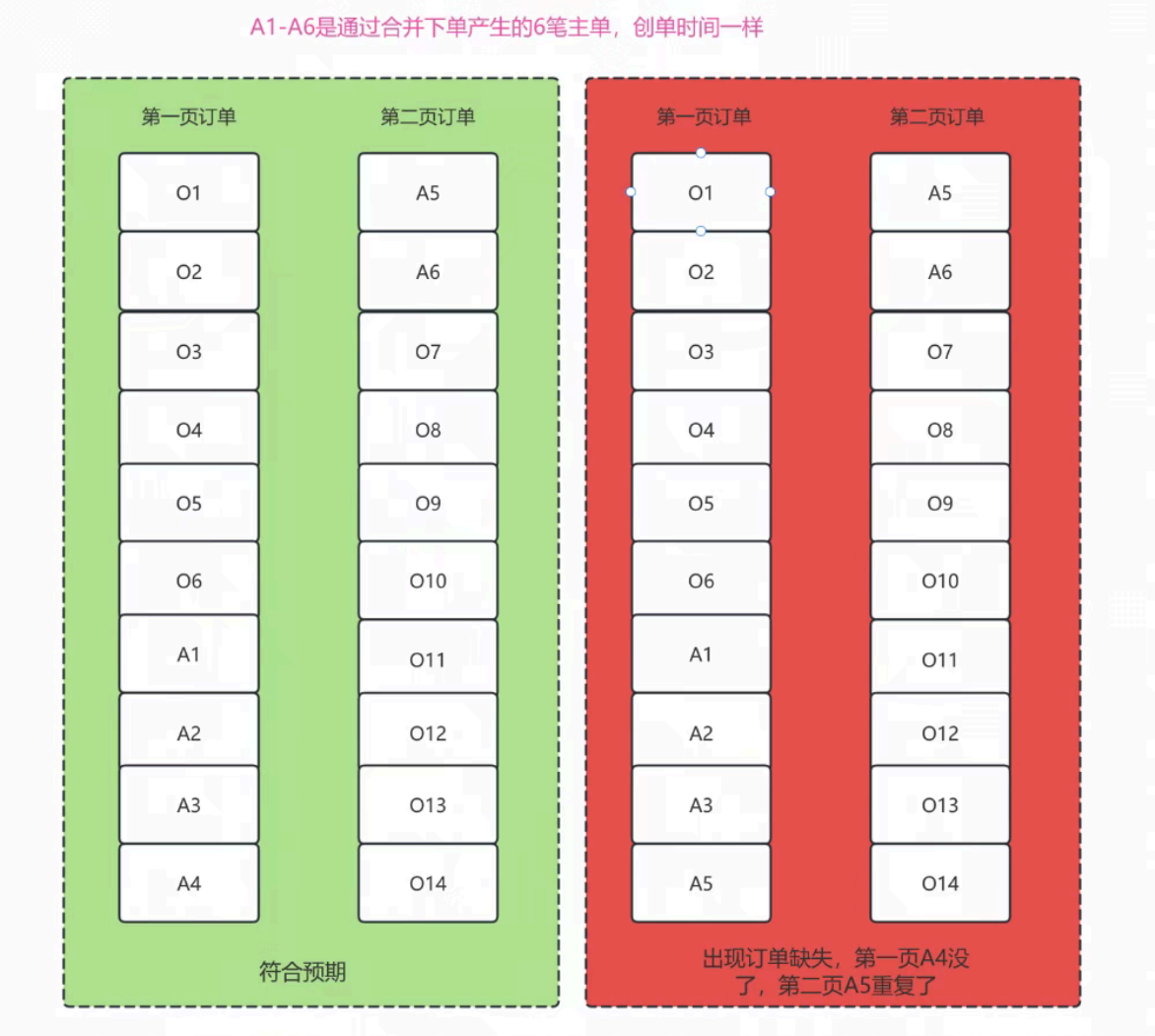


同一个SQL，对比create_time desc排序（query 3）和create_time desc,order_id asc排序（query 2），可以发现前者利用索引排序（Sorting result），耗时很低0.000019，后者走文件堆排（Creating sort index），耗时很高0.094799。

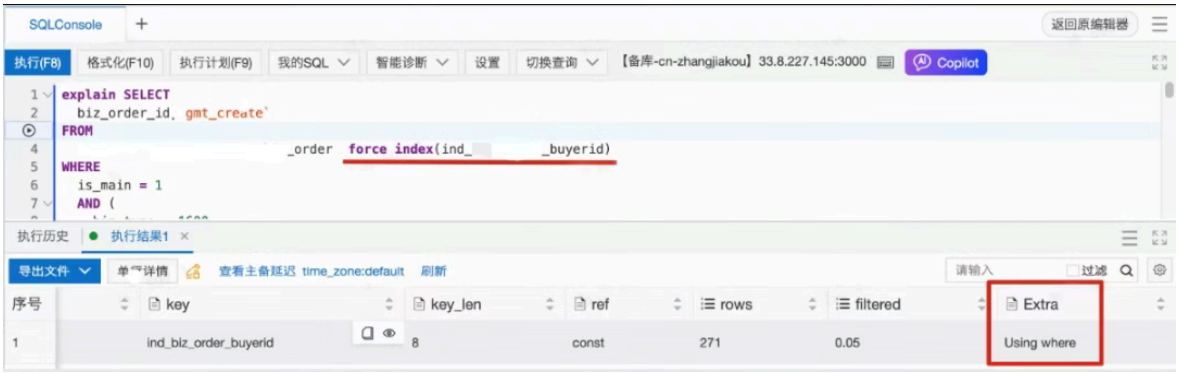


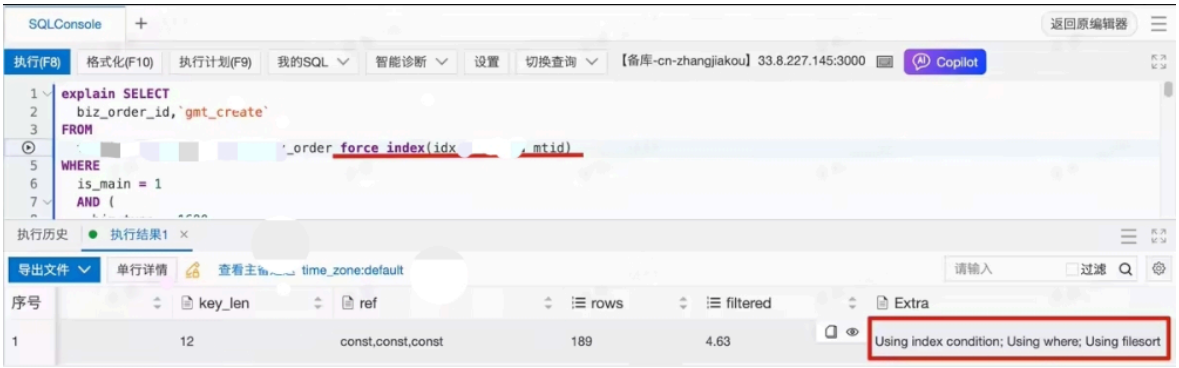
为了解决上述慢SQL，最简单的办法就是把order_id排序去掉，不过在此之前先回顾下增加order_id排序的背景。24年做订单列表后置过滤治理时，tm2在订单列表查询条件中新增了一个m_tid字段，该字段用于过滤掉不在手淘上展示的订单，从而实现在DB侧前置过滤掉不在手淘上展示的订单，不用后置到tm2内存过滤。不过m_tid字段没有索引，为保证压测通过，新增了索引`idx_***_mtid`(`buyer_id`,`is_main`,**********,`create_time`,**********)。因为create_time是排序条件，不是查询条件，所以新索引把create_time字段放到索引中倒数第三的位置上。

基于新索引，DB单实例压测和全链路压测都没问题，但是tm2在放量结束后的第二天上午突然出现了大量订单找不到或者订单重复的舆情，典型的case如下：



在dms上通过force index指定索引的方式复现了上述问题，发现走老索引没问题，走新索引有问题，二者的explain差异如下：





跟DBA沟通确认，走老索引ind_***_buyerid时，因为create_time字段在索引中第二个字段，create_time desc排序可以直接利用索引排序，返回的订单ID是按照索引里面固定的顺序返回。走新索引idx_****_mtid时，因为create_time是索引中倒数第三个字段，create_time desc排序只能走文件堆排，而第一页订单请求和第二页订单请求的文件堆排的排序结果可能不同，即A1–A6的顺序是不确定的，一旦两次排序顺序不同，计算分页时获取的订单就可能出现重复或者订单缺失的问题，比如第一页订单请求时排序是A1,A2,A3,A5,A4,A6,取前面4条，第二页订单请求时排序是A1,A2,A3,A4,A5,A6，取后面2条，即出现A4缺失，A5重复的问题。因为索引二次变更没有足够的时间窗口，当时决策采用风险相对较小的方案，把排序条件从create_time desc改成create_time desc,order_id asc排序，通过order_id asc来保证创单时间一致的情形下返回的订单列表的稳定。因为二级索引的叶子节点里面本身是包含有order_id字段的，所以不会新增回表查，对DB的影响主要是新增order_id排序本身的损耗，通过DB单实例压测验证这部分损耗对整体CPU水位的影响不明显，后续也通过了双11考验。

明确增加order_id排序的原因后，就制定了一种风险相对较小的优化方式，把非tm2订单列表即不带m_tid字段的分页查询请求中的order_id排序去掉，预期这类SQL应该走老索引ind_***_buyerid，从而保证返回的订单列表的稳定。放量完成后发现不带m_tid字段的分页查询请求也会走到新索引 idx_****_mtid，如下：



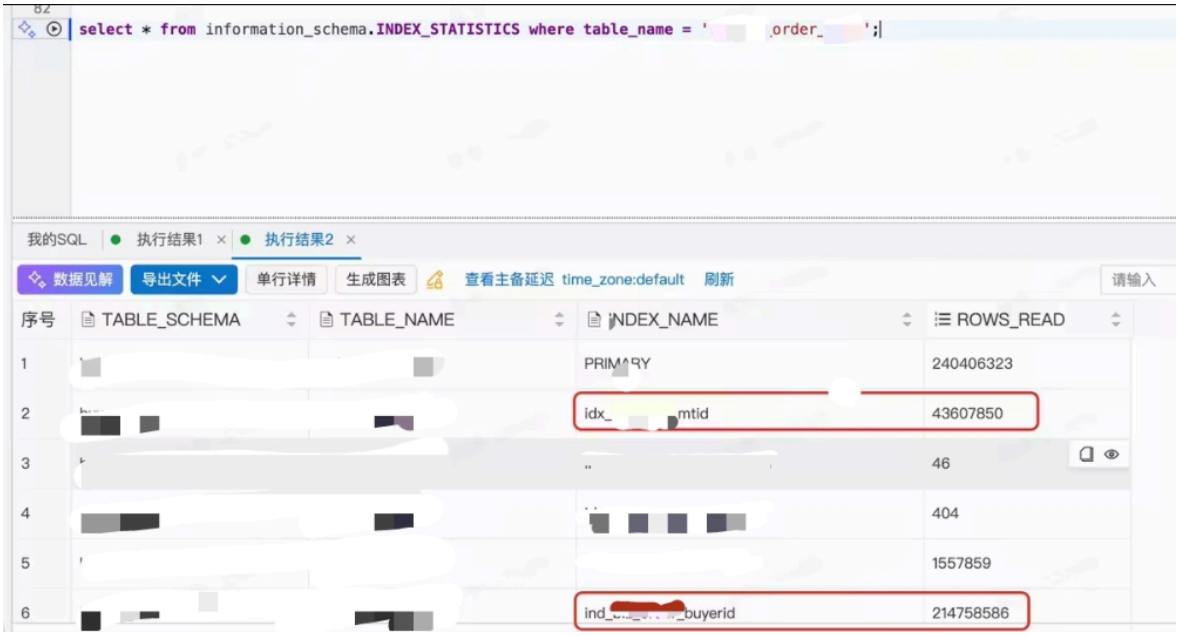
查询条件中带有create_time的范围查询，理论来说走老索引ind_***_buyerid更优，可以走索引过滤而不是索引下推过滤。实际测试发现当查询时间是2025–08–01会走老索引ind_***_buyerid，如下：



该用户下创单最早的订单都是大于2025-01-01，初步怀疑create_time大于2025-01-01的查询条件被索引选择器直接忽略。如果把查询时间去掉，也是走新索引 idx_****_mtid，相比老索引可以有更多的字段走索引过滤，符合预期。



可以通过INFORMATION_SCHEMA.INDEX_STATISTICS表 来查看走各个索引查询返回的记录数：

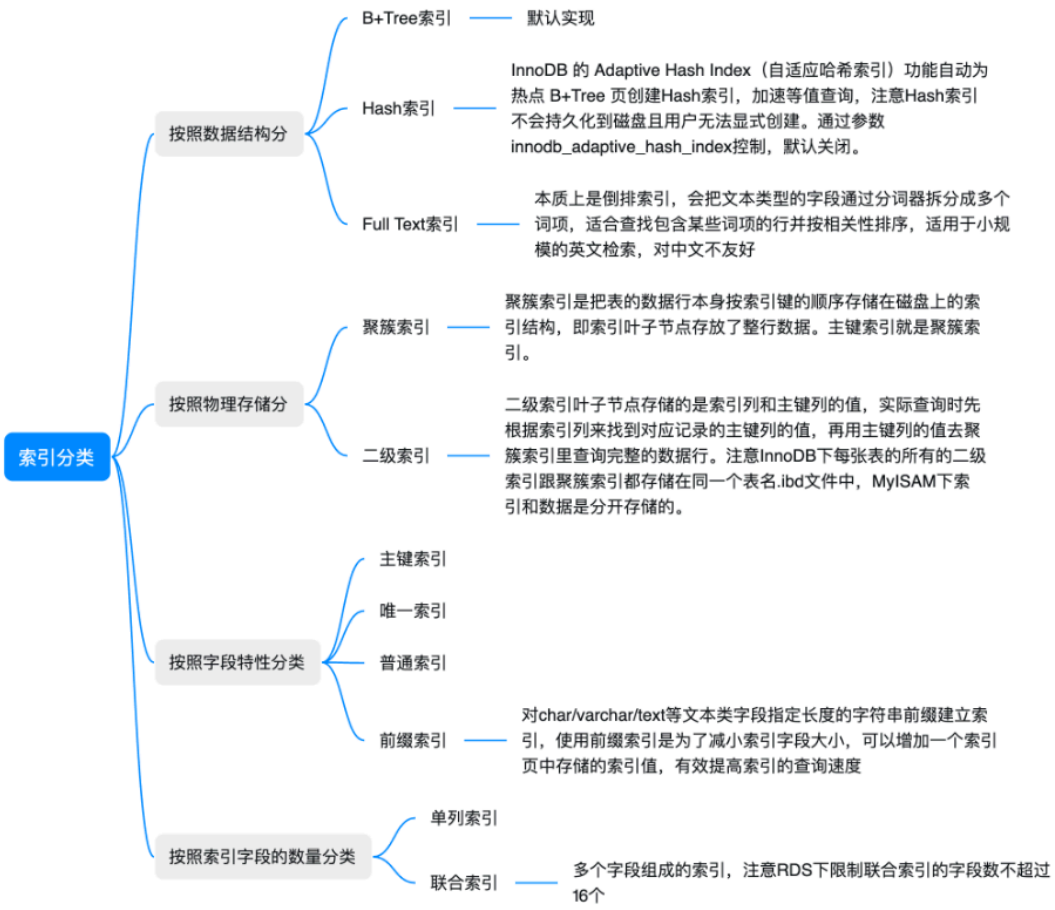


基于上述统计结果可知，实际查询中有很多case依然走新索引idx_****_mtid。为了彻底解决新索引带来的排序问题，同时解决业务要求的新的查询字段加索引问题，决策再次调整订单表的索引。

04 索引知识回顾

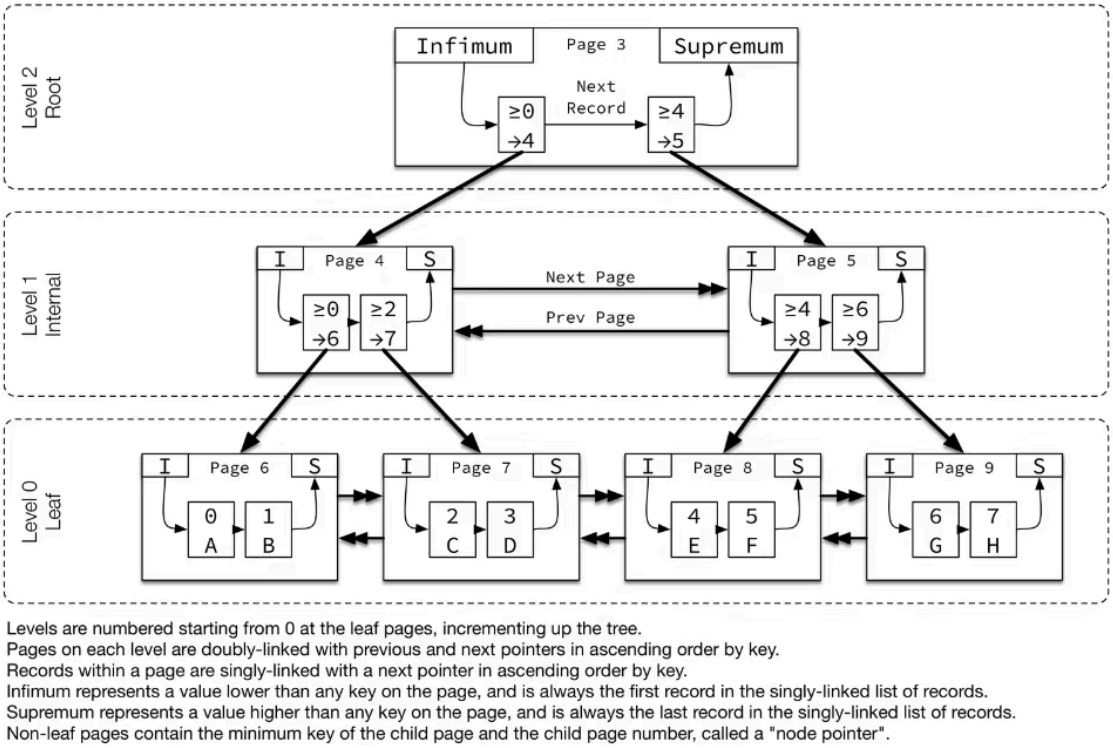
磨刀不误砍柴工，先系统回顾总结下相关的知识点，打好理论基础。

索引分类



B+Tree数据结构

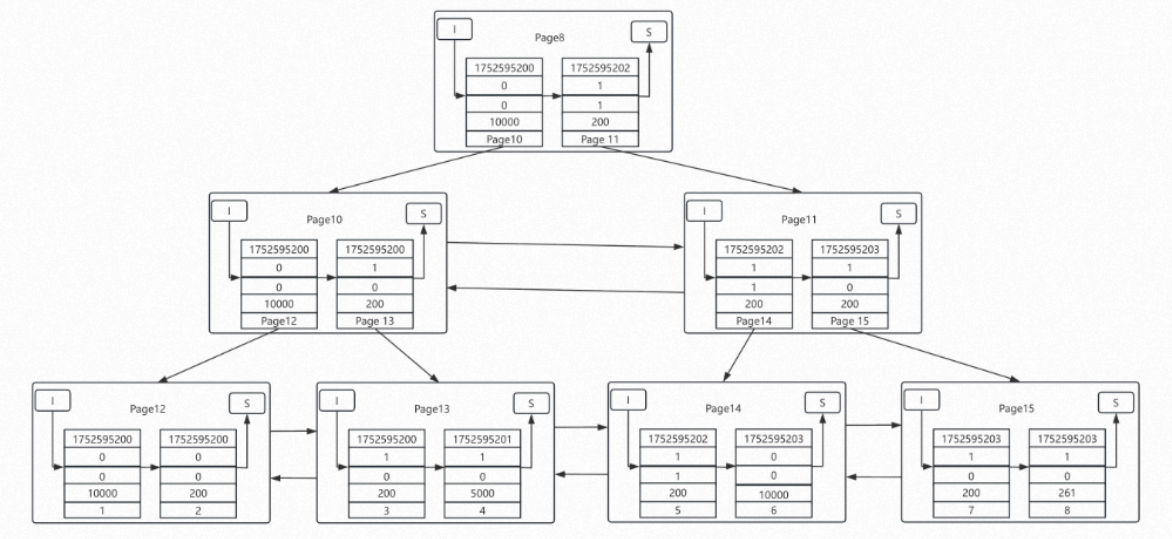
找了一圈资料，发现只有下面这张图是比较清晰和准确的，下面基于这个图来做补充。



B+Tree是一个多叉树，即子节点的数量可以大于2，与之相对的红黑树最多只有2个子节点。B+Tree中一个节点的子节点数目的最大值就是B+Tree的阶，比如8阶B+Tree最多有8个子节点。B+Tree通过增加子节点的数量可以大幅增加单层B+Tree的节点数（比如3层8阶的B+Tree第三层的节点数等于8*8*8=252，3层的红黑树第三层的节点数等于2*2*2=8），从而大幅降低整体的树高度，减少查询过程中的磁盘访问，提升读写效率，比如图中树的高度是3，从根节点到叶子节点只需要3次磁盘访问即可返回。

B+Tree中单个节点实际对应一个16kb的物理Page，Page的Fil Header中会维护两个链表指针，指向相邻的两个Page，通过这两个指针，每一层的Page都会组成一个横向的Page双向链表，从而支持双向遍历，提升列表查询效率。每个Page中会包含多条索引记录Record，各个Record在逻辑上基于索引列值升序排列并组成一个单向链表，这些称为User Records。Page内预留了两个固定位置固定值的System Records，在链表头的Infimum记录以及在链表尾的Supremum记录，Infimum指向索引值最小的Record，Supremum通常为null。Page内部还会维护一组Directory slot，每个Directory slot指向一个Record记录，两个Directory slot对应的Record记录中间会相隔4到8个Record记录，实际在Page内部查找满足条件的Record记录时先根据Directory slot做二分查找找到相邻的两个slot记录，再顺序遍历这两个slot中间的Record记录，从而提升查找效率。

B+Tree中的节点分为三种，根节点、叶子节点和中间的非叶子节点，其中主键索引根节点对应PageNo默认是3，可以根据PageNo和PageSize计算出文件偏移量，再根据偏移量从idb文件中读取对应PageSize的字节数据，再按照约定的数据存储格式即可解析出来存储的原始数据。三种节点包含的Record的Header部分基本一致，主要区别是Data部分。其中根节点和非叶子节点记录的Data是索引字段的值和指向下层子节点的PageNo，如果是主键索引则只有一个索引字段，如果是二级索引则可能存在多个索引字段，注意索引字段的值是下层子节点中所有Record中最小的且每个Record都对应一个下层节点，比如图中Page4 Record为0的记录指向Page6，包含0和1两条记录，Record为2的记录指向Page7，包含2和3两条记录。对于叶子节点包含的Record，如果是主键索引则Data是索引字段的值以及其他的非索引字段的值，这两者加起来构成完成的一行数据，比如图中的A,B,C,D,E,F,G,H就是非索引字段的取值；如果是二级索引则Data是二级索引各个字段的值加上主键索引字段的值，如下图：



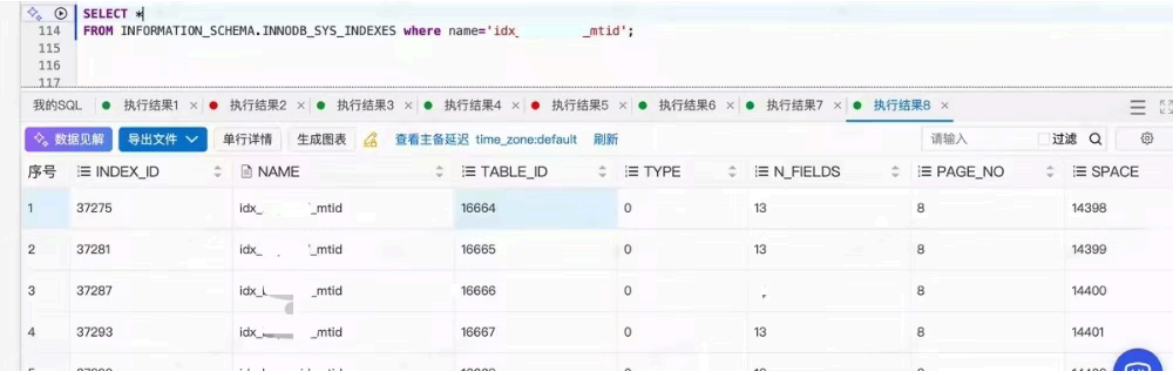
对应的原始数据如下：

order_id	create_time	is_main	status	type
1	1752595200	0	0	10000
2	1752595200	0	0	200
3	1752595200	1	0	200
4	1752595201	1	0	5000
5	1752595202	1	1	200
6	1752595203	0	0	10000

7	1752595203	1	0	200
8	1752595203	1	0	261

二级索引顺序是按照多个索引字段的顺序从左往右依次比较，跟比较两个字符串的顺序的规则基本一致，比如上面的case，先比较create_time，create_time一致的情况下再比较is_main，is_main一致的情况下再比较status，这样设计会导致二级索引只有第一个字段是有序的，其他的字段实际存储都是无序的；如果二级索引前面的字段的值是确定的，则后面的第一个字段的存储也是有序的。

二级索引的Root Page的PageNo是通过索引的元数据表INFORMATION_SCHEMA.INNODB_SYS_INDEXES 维护的，如下：



B+Tree的高度测算

单个Page包含的Record记录数取决于单个Record的大小，以tcorder为例分析，主键索引是order_id，BIGINT类型，占8个字节，非叶子节点的Record的大小约为8+5（Header）+4(子节点PageNo指针)=13，单个Page包含的最大的Record数量为（16*1024-200（Header））/13=952。叶子节点的大小为单行的行长5.4kb+20（Header）=5.5kb，单个Page包含的最大的Record数量是2。据此不同高度的B+Tree支持的最大记录数如下：

高度	非叶子节点数量	叶子节点数量	最大记录数
1	0	1	2
2	1	952	952*2=1904

3	1+952=953	952*952=906304	952*952*2=1812608
4	1+952+952*952=907257	952*952*952=862801408	952*952*952*2=1725602816

索引`ind_***_buyerid` 的长度是58，非叶子节点的Record的长度约是58+5+4=67，单个Page包含的最大的Record数量为（16*1024−200（Header））/67=241。叶子节点的Record的长度是67+8（主键索引）+20（Header）=95，单个Page包含的最大的Record数量是（16*1024−200（Header））/95=170个。据此不同高度的B+Tree支持的最大记录数如下：

高度	非叶子节点数量	叶子节点数量	最大记录数
1	0	1	170
2	1	241	241*170=1904
3	1+241=242	241*241=58081	241*241*170=9873770
4	1+241+241*241=58323	241*241*241=13997521	241*241*241*170=2379578570

通过上述测算可知：

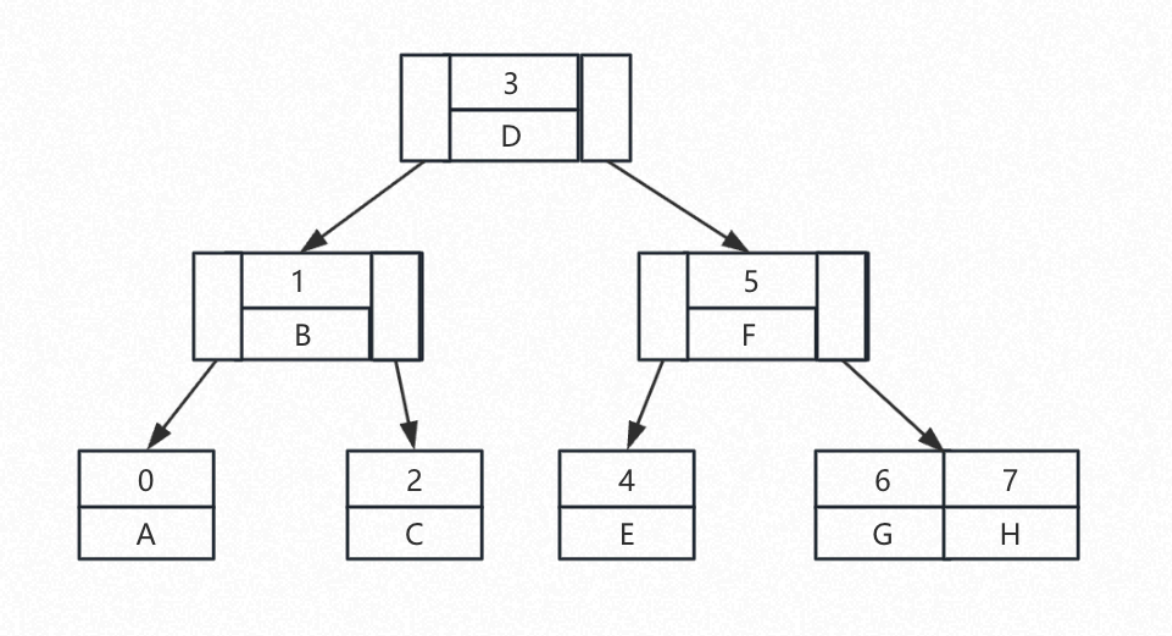
1. tcorder单表的量级在800w左右，按照上面的测算，主键索引B+Tree的高度是4，二级索引B+Tree的高度是3。
2. 在存储同等量级的数据的前提下，行长和索引长度增加会导致单个Page包含的Record数量减少，导致B+Tree的高度增加，导致读写效率下降，所以需要限制单表的字段数量和索引的字段数量。
3. 对于二级索引，索引长度不超过50，单表控制在1000w行以内，B+Tree的高度不超过3。
4. 对于主键索引，行长不超过1.5kb，单表控制在1000w行以内，B+Tree的高度不超过3。

B-Tree的数据结构

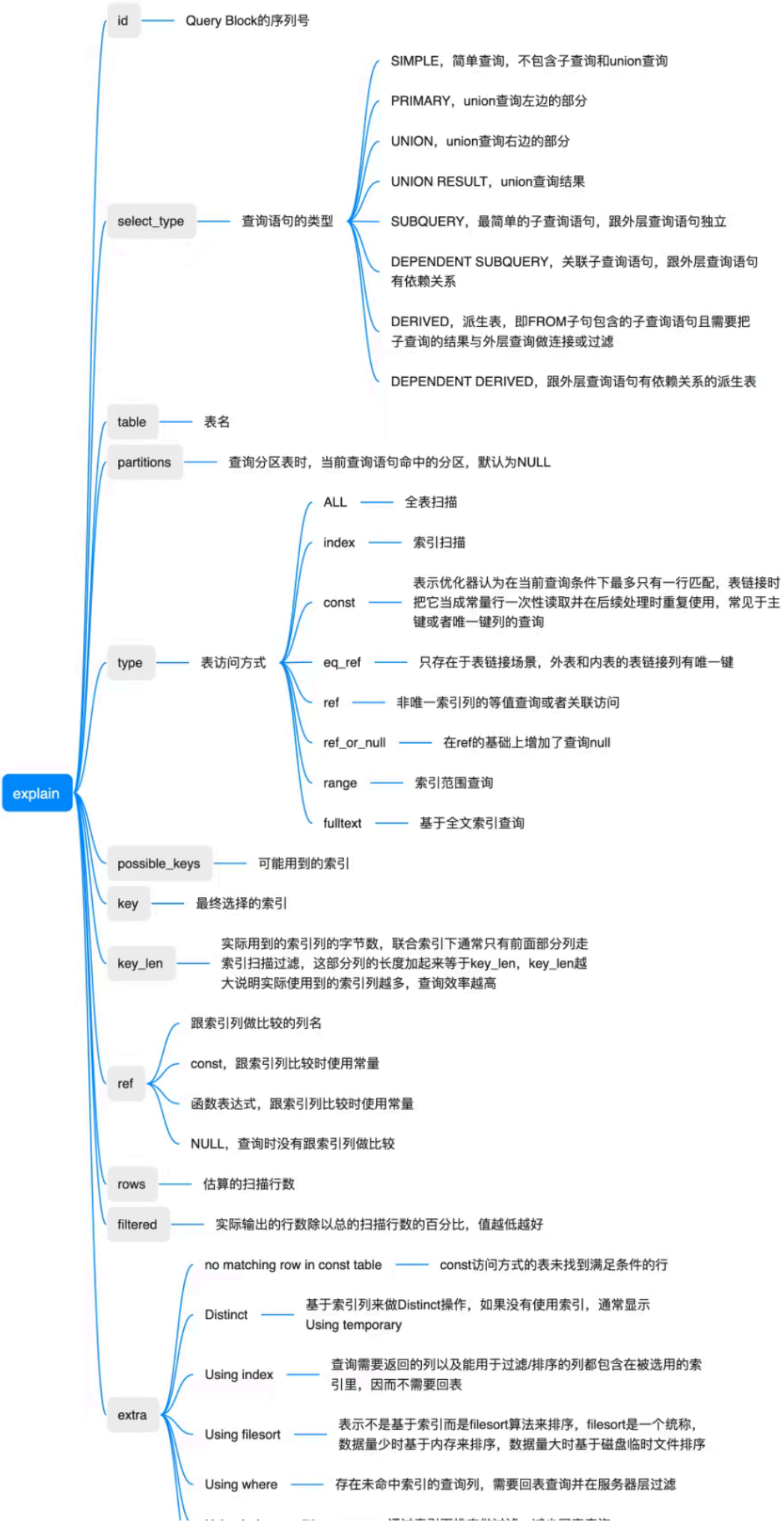
B+Tree是从B-Tree改进而来，两者的区别如下：

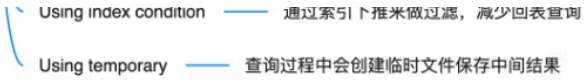
	m阶B-Tree	m阶B+Tree
子节点数量	m+1	m
非叶子节点是否包含非索引字段数据	是，任何索引记录只存在于一个节点中	否，非叶子节点只存储索引字段，只用于二分查找，因此同一个索引记录可能出现在多个非叶子节点中，只有叶子节点保存了完整的索引数据
同一层的节点是否有双向指针	否	是
优势	没有数据冗余，数据规模较小时，因为命中了某个非叶子节点即可返回，点查场景下查询效率更高	在大规模数据下因为非叶子节点包含的索引记录更多，从而降低整体B+Tree的高度，提升查找效率。 每次查询必须走到叶子节点，从而保证查询RT的稳定。 同一层的节点有双向指针，提高列表查询的效率。
缺点	每次查询命中的节点的树高度可能不一致导致查询RT不稳定 同一层的节点没有双向指针，不利于列表查询。	有数据冗余

同样的数据，B-Tree的数据结构如下：



explain说明


explain
id
Query Block的序列号
select_type
查询语句的类型
SIMPLE，简单查询，不包含子查询和union查询
PRIMARY，union查询左边的部分
UNION，union查询右边的部分
UNION RESULT，union查询结果
SUBQUERY，最简单的子查询语句，跟外层查询语句独立
DEPENDENT SUBQUERY，关联子查询语句，跟外层查询语句有依赖关系
DERIVED，派生表，即FROM子句包含的子查询语句且需要把子查询的结果与外层查询做连接或过滤
DEPENDENT DERIVED，跟外层查询语句有依赖关系的派生表
table
表名
partitions
查询分区表时，当前查询语句命中的分区，默认为NULL
type
表访问方式
ALL
全表扫描
index
索引扫描
const
表示优化器认为在当前查询条件下最多只有一行匹配，表链接时把它当成常量行一次性读取并在后续处理时重复使用，常见于主键或者唯一键列的查询
eq_ref
只存在于表链接场景，外表和内表的表链接列有唯一键
ref
非唯一索引列的等值查询或者关联访问
ref_or_null
在ref的基础上增加了查询null
range
索引范围查询
fulltext
基于全文索引查询
possible_keys
可能用到的索引
key
最终选择的索引
key_len
实际用到的索引列的字节数，联合索引下通常只有前面部分列走索引扫描过滤，这部分列的长度加起来等于key_len，key_len越大说明实际使用到的索引列越多，查询效率越高
ref
跟索引列做比较的列名
const，跟索引列比较时使用常量
函数表达式，跟索引列比较时使用常量
NULL，查询时没有跟索引列做比较
rows
估算的扫描行数
filtered
实际输出的行数除以总的扫描行数的百分比，值越低越好
extra
no matching row in const table
const访问方式的表未找到满足条件的行
Distinct
基于索引列来做Distinct操作，如果没有使用索引，通常显示Using temporary
Using index
查询需要返回的列以及能用于过滤/排序的列都包含在被选用的索引里，因而不需要回表
Using filesort
表示不是基于索引而是filesort算法来排序，filesort是一个统称，数据量少时基于内存来排序，数据量大时基于磁盘临时文件排序
Using where
存在未命中索引的查询列，需要回表查询并在服务器层过滤



explain的输出默认是表格模式的，还有一个json格式，输出的信息更详尽，通过explain format=json SQL的方式获取，以这个SQL为例：

```
select
  ********
from
  (
    select
      order_id
    from
      `tcorder`
    where
      is_main = 1
      and buyer_id = 1*******1
      **************
    order by
      create_time desc,`order_id` asc
    limit
      0, 10
  ) t2,
  `tcorder` t1
where
  t2.order_id = t1.order_id
```

对应的表格输出如下，输出的表的顺序并不是实际的执行顺序，而是先外层表再内层表。

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	<derived2>		ALL					5	100.00	
1	PRIMARY	t1		eq_ref	PRIMARY	PRIMARY	8	t2.order_id	1	100.00	
2	DERIVED	tcorder		ref	ind_***_buyerid,idx_***_gmtcreate	idx_***_gmtcreate	8	const	16982	0.03	Using index condition; Using where; Using filesort

对应的json输出如下：

```
{
  "query_block": {--最外层的SQL的执行成本
    "select_id": 1,
    "cost_info": {
      "query_cost": "17.25"--预估的成本值，不是预估耗时，只能用作横向比较
    },
```

```
    "nested_loop": [--按照执行顺序列出来多个内层SQL预估的执行成本
      {
        "table": {
          "table_name": "t2",
          "access_type": "ALL",
          "rows_examined_per_scan": 5,--预估的扫描行数
          "rows_produced_per_join": 5,--预估的返回上层的行数
          "filtered": "100.00",
          "cost_info": {
            "read_cost": "10.25",--预估的单次读取数据的成本
            "eval_cost": "1.00",--预估的将单次读取的数据按照where条件过滤一
            "prefix_cost": "11.25",--整体的查询成本
            "data_read_per_join": "80"--预估读取的数据量，单位字节
          },
          "used_columns": [
            "order_id"
          ],
          "materialized_from_subquery": {--表示t2是物化子查询产生的
            "using_temporary_table": true,--使用了临时表保存结果
            "dependent": false,--表示子查询的where条件跟外层SQL无关，在外层
            "cacheable": true,--表示子查询的结果可以缓存
            "query_block": {
              "select_id": 2,
              "cost_info": {
                "query_cost": "20928.00"
              },
              "ordering_operation": {--表示需要排序
                "using_filesort": true,--表示走文件堆排排序
                "table": {
                  "table_name": "tcorder",
                  "access_type": "ref",
                  "possible_keys": [
                    "ind_***_buyerid",
                    "idx_***_gmtcreate"
                  ],
                  "key": "idx_***_gmtcreate",
                  "used_key_parts": [--表示实际生效的索引列
                    "buyer_id"
                  ],
                  "key_length": "8",
                  "ref": [
                    "const"
                  ],
                  "rows_examined_per_scan": 17440,
                  "rows_produced_per_join": 5,
                  "filtered": "0.03",
```

```
                    --index_condition表示通过索引下推过滤的查询条件
                    "index_condition": "
                    ****************
                    ((`tcorder`.`buyer_id` <=> 117075031) and (``tcorde
                    ***************
                    ",
                    "cost_info": {
                      "read_cost": "17440.00",
                      "eval_cost": "1.18",
                      "prefix_cost": "20928.00",
                      "data_read_per_join": "236K"
                    },
                    "used_columns": [
                      "order_id",
                      "buyer_id",
                      *************
                    ],
                    --attached_condition表示通过mysql服务器层过滤的where条件
                    "attached_condition": "((ifnull(`tcorder`.`attribute
                  }
                }
              }
            }
          }
        },
        {
          "table": {
            "table_name": "t1",
            "access_type": "eq_ref",
            "possible_keys": [
              "PRIMARY"
            ],
            "key": "PRIMARY",
            "used_key_parts": [
              "order_id"
            ],
            "key_length": "8",
            "ref": [
              "t2.order_id"
            ],
            "rows_examined_per_scan": 1,
            "rows_produced_per_join": 5,
            "filtered": "100.00",
            "cost_info": {
              "read_cost": "5.00",
              "eval_cost": "1.00",
```

```
            "prefix_cost": "17.25",
            "data_read_per_join": "200K"
          },
          "used_columns": [
            "order_id",
            ****************
          ]
        }
      }
    ]
  }
}
```

综合上面json的输出可知上述SQL的实际执行过程大致如下：

1. 先执行子查询（select_id=2），扫描 tcorder 的索引 idx_***_gmtcreate（约 17440 条候选），应用 index_condition（ICP）以减少要回表的行数；仍需对 attached_condition 做服务器端判断；对结果做排序（using_filesort），把结果物化到临时表（using_temporary_table=true）。物化后临时表（t2）约有 5 行（非常小且 cacheable，因为 dependent=false）。
2. 外层扫描临时表 t2（ALL，5 行），对每个 t2.order_id 用主键在 t1 上做 eq_ref 点查（一次找到一行），最终组合出 5 条输出行。

Query Profiler

MySQL 的 Query Profiler 是一个使用非常方便的 Query 诊断分析工具，通过该工具可以获取一条Query 在整个执行过程中不同阶段多种资源的消耗情况，如耗时、CPU，IO，IPC，SWAP 等，以及发生的 PAGE FAULTS，CONTEXT SWITCHE次数等等，同时还能得到该 Query 执行过程中 MySQL 所调用的各个函数在源文件中的位置，其具体用法如下：

命令	说明	示例
show variables like 'profiling'	查看是否开启profile功能，默认关闭	/** 查看是否开启 Query Profiler */ mysql> show variables like 'profiling' ; +---------------+-------+ | Variable_name | Value | +---------------+-------+ | profiling | OFF | +---------------+-------+
set profiling = 1	开启Query Profiler功能，开启后执行任何SQL都会被采集Profiler信息	

show profiles	–查看当前保留的采集了profile信息的SQL列表，默认是最多保存15个，通过参数profiling_history_size控制	
SHOW PROFILE [type] [FOR QUERY n] [LIMIT row_count [OFFSET offset]]	查看具体的profile信息，type取值可以是 ALL\BLOCK IO等，多个type之间用逗号间隔。FOR QUERY n中的n是SHOW PROFILES中的编号，可以查看指定缓存的Profiling信息。LIMIT OFFSET控制的是一条查询的Profiling信息中输出哪部分行（阶段）的信息。直接执行SHOW PROFILE展示的是最新一条被缓存的Profiling信息	

type的取值如下：

取值	说明
ALL	显示所有的开销信息
BLOCK IO	显示块IO相关开销
CONTEXT SWITCHES	显示上下文切换相关开销
CPU	显示CPU相关开销信息
IPC	显示发送和接收相关开销信息
MEMORY	显示内存相关开销信息，5.7版本未实现
PAGE FAULTS	显示页面错误相关开销信息
SOURCE	显示和Source_function，Source_file，Source_line相关的开销信息

SWAPS	显示交换次数相关开销的信息

Profiler采集的信息最终保存在INFORMATION_SCHEMA.PROFILING表中，相关字段说明如下：

取值	说明
QUERY_ID	查询语句的ID
STATE	具体的状态，对应输出结果的status列
Duration	状态总持续时间，单位秒
CPU_user	用户CPU时间，单位秒
CPU_system	系统CPU时间，单位秒
Context_voluntary	上下文主动切换次数
Context_involuntary	上下文被动切换次数
Block_ops_in	磁盘阻塞写入次数
Block_ops_out	磁盘阻塞读取次数
Messages_sent	发送的消息量
Messages_received	接收的消息量
Page_faults_major	主要页面错误的数量
Page_faults_minor	次要页面错误的数量
Swaps	交换次数
Source_function	原函数（调用源码中的函数）
Source_file	源文件（MySQL源码中对应的.cc文件）

Source_line	源代码行（对应源码文件中的代码行数）

status的取值如下：

取值	说明
starting	MySQL 为执行该语句做初始准备（启动执行流程）。
checking permissions（或 checking permissions on table）	检查用户权限，确认是否有访问表/列的权限。
Waiting for query cache lock	等待查询缓存的锁（如果启用了 query_cache），常见于并发环境
Opening tables	打开表（打开文件/表句柄、表定义等），这是执行前的准备。
After opening tables / After opening tables	表示已完成打开表的步骤（有时重复出现）。
System lock	获取全局/系统级锁（各种内部同步），旧存储引擎或元数据操作时可见。
Waiting for table metadata lock/Waiting for metadata lock / Waiting for table lock	等待元数据锁（MDL），例如其他会话正在做 ALTER/LOCK TABLE 等。
init	初始化执行环境（分配内存、初始化变量等）。
optimizing / optimizing (or optimise)	优化器进行重写/选择索引并生成访问计划。
statistics	收集统计信息（表、索引基数、列分布等）以估算成本（优化器用这些信息）。
preparing	为执行做最后准备（例如为执行计划的某些部分生成临时结构）。

Sending data	通常表示把结果行从存储引擎/服务器发送到客户端、或是执行过程中从存储引擎读取并向上层传递行。
executing	表示运行语句的服务器端逻辑（包括存储过程/函数/触发器的代码、表达式求值、调用存储引擎的执行逻辑等），这是一个比较通用的执行阶段标识。
Copying to tmp table	将结果复制到临时表（当需要 GROUP BY、DISTINCT、某些 ORDER BY、子查询物化时）。这是将数据写入临时表的阶段。
Creating tmp table	创建临时表结构（内存或磁盘）以便后续使用。
Sorting for group / Sorting for order / Sorting result	对数据做排序（filesort），可能用于 GROUP BY、ORDER BY。filesort 多时开销大。
Creating sort index / Repair by sorting	与索引重建或修复相关（不常见于普通查询，常见于 REPAIR 操作）。
Waiting for table level lock / Waiting for table flush	等待表级锁或表刷新相关
Sorting for group	为 GROUP BY 排序或准备聚合（可能和 loose index scan 相关）
end / query end	查询逻辑结束，进入收尾阶段。
closing tables	关闭已打开的表/文件句柄。
freeing items	释放表达式/缓存所占资源（清除内存）。
cleaning up	最终清理，回收分配、释放临时表等。

各个阶段的准确含义可以根据打日志对应的源码位置翻阅源码确认。根据各个阶段的耗时情况可以分析某个慢SQL的问题根因，比如排序耗时占比过高则需要优化排序逻辑或者给排序字段

加索引。注意Query Profiler在mysql 8.0以上版本已经废弃，可以使用功能更强大的EXPLAIN ANALYZE功能来分析单个SQL的实际执行过程。

索引下推

先看看没有索引下推的情形，如下SQL：

```
explain select
  *
from
  `tcorder`
where
  `parent_id` = 37*******0834
  and `from_group` = 3;
```

输出结果：

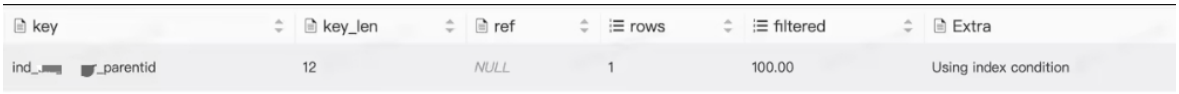
key	key_len	ref	rows	filtered	Extra
ind_ _parentid	12	const,const	1	100.00	NULL

据此可知，上述SQL完全命中的索引ind_***_parentid(parent_id,from_group)且通过const常量方式查询索引，最多命中一行记录，此时是存储引擎根据索引找到符合条件的记录后再回表获取完整的记录行。因为是const常量访问，此时Extra为NULL，没有显示Using Index。

调整查询条件from_group为大于，如下：

```
explain select
  *
from
  `tcorder`
where
  `parent_id` = 37*******0834
  and `from_group` > 3;
```

输出结果：



key	key_len	ref	rows	filtered	Extra
ind_[redacted]_parentid	12	NULL	1	100.00	Using index condition

json格式的结果如下：

```
{
  "query_block": {
    "select_id": 1,
    "cost_info": {
      "query_cost": "1.42"
    },
    "table": {
      "table_name": "tcorder",
      "access_type": "range",
      "possible_keys": [
        "ind_***_parentid"
      ],
      "key": "ind_***_parentid",
      "used_key_parts": [
        "parent_id",
        "from_group"
      ],
      "key_length": "12",
      "rows_examined_per_scan": 1,
      "rows_produced_per_join": 0,
      "filtered": "100.00",
      "index_condition": "((`tcorder`.`parent_id` = 37*******0834) and
      "cost_info": {
        "read_cost": "1.18",
        "eval_cost": "0.08",
        "prefix_cost": "1.42",
        "data_read_per_join": "15K"
      },
      "used_columns": [
        "order_id",
        ***************
      ]
    }
  }
}
```

Using index condition表示使用了索引下推（index condition pushdown，简称ICP），json输出中index_condition表明parent_id和from_group两个条件都通过索引下推来做过滤了。

假设满足parent_id条件的有10条，同时满足两个条件的只有1条，不开启ICP时，整体的查询过程如下：

1. 存储引擎读取满足parent_id条件的10条索引记录。
2. 存储引擎根据索引记录中的主键ID读取10行完整的行记录。
3. 存储引擎返回给mysql server层，由server层完成剩余的from_group条件的过滤并返回给客户端。

开启ICP时，整体的查询过程如下：

1. 存储引擎读取满足parent_id条件的10条索引记录，利用索引中记录的from_group字段的值来做from_group条件的过滤，并最终返回1条索引记录。
2. 存储引擎根据索引记录中的主键ID读取1行完整的行记录。
3. server层直接把存储引擎返回的1行记录返回给客户端。

整个查询过程中回表的次数就从10次降低为1次，从而提升整体的效率。ICP是默认开启的（通过参数index_condition_pushdown控制），为了充分发挥ICP的优化效果，需要把区分度大的字段尽可能加入到二级索引中，从而大幅降低回表查的次数。

查询的字段在二级索引中，命中下列情形时也可能不使用ICP:

1. 如果索引已覆盖所需返回列，无需回表，Extra显示Using index。
2. 为了满足 ORDER BY/LIMIT，优化器可能选择按索引顺序扫描或 filesort。
3. 查询条件中包含非确定性/有副作用函数，比如now()/RAND()等，必须在server层执行过滤。
4. 优化器估算的候选条目很少导致下推收益小，优化器可能选择在服务器层过滤。

比如下面的SQL：

```
select
      order_id
    from
      `tcorder`
    where
      is_main = 1
      and buyer_id = 117075031
      ***************
      -- and (
```

```
      --   ifnull(attribute4, 0) <> 1
      --   and ifnull(attribute4, 0) <> 2
      -- )
      ***************
    order by
      create_time desc
    limit
      0, 10;
```

注掉attribute4条件，所有的查询字段和返回字段都在索引里面有，不需要回表，explain结果如下：

key	key_len	ref	rows	filtered	Extra
idx_ gmtcreate	8	const	17266	0.03	Using where; Using index

执行过程：通过mysql server层来完成索引记录的过滤，从索引记录中读取order_id后直接返回，不回表。

恢复attribute4条件，因为attribute4不在索引里，需要回表查，所以不显示Using index，如下：

key	key_len	ref	rows	filtered	Extra
idx_ _gmtcreate	8	const	17266	0.03	Using where

执行过程：server层完成索引记录的过滤后回表查完整的记录行，再执行attribute4的过滤。

去掉order by条件，显示Using index condition，如下：

key	key_len	ref	rows	filtered	Extra
idx_ gmtcreate	8	const	17266	0.03	Using index condition; Using where

执行过程：除attribute4外在存储引擎层过滤，过滤完成后回表查，在server层完成attribute4字段的过滤。

排序优化

mysql处理order by时先判断能否使用索引来做排序，判断标准是排序条件必须跟索引B+Tree叶子节点的排序完全一致。因为索引B+Tree叶子节点中的数据本身就是排序好的，基于索引顺序或者倒序读取出来的记录可以直接返回给上层，不需要额外的排序操作。以索引ind_***_parentid(parent_id,from_group)为例说明，如下情形可以利用索引排序：

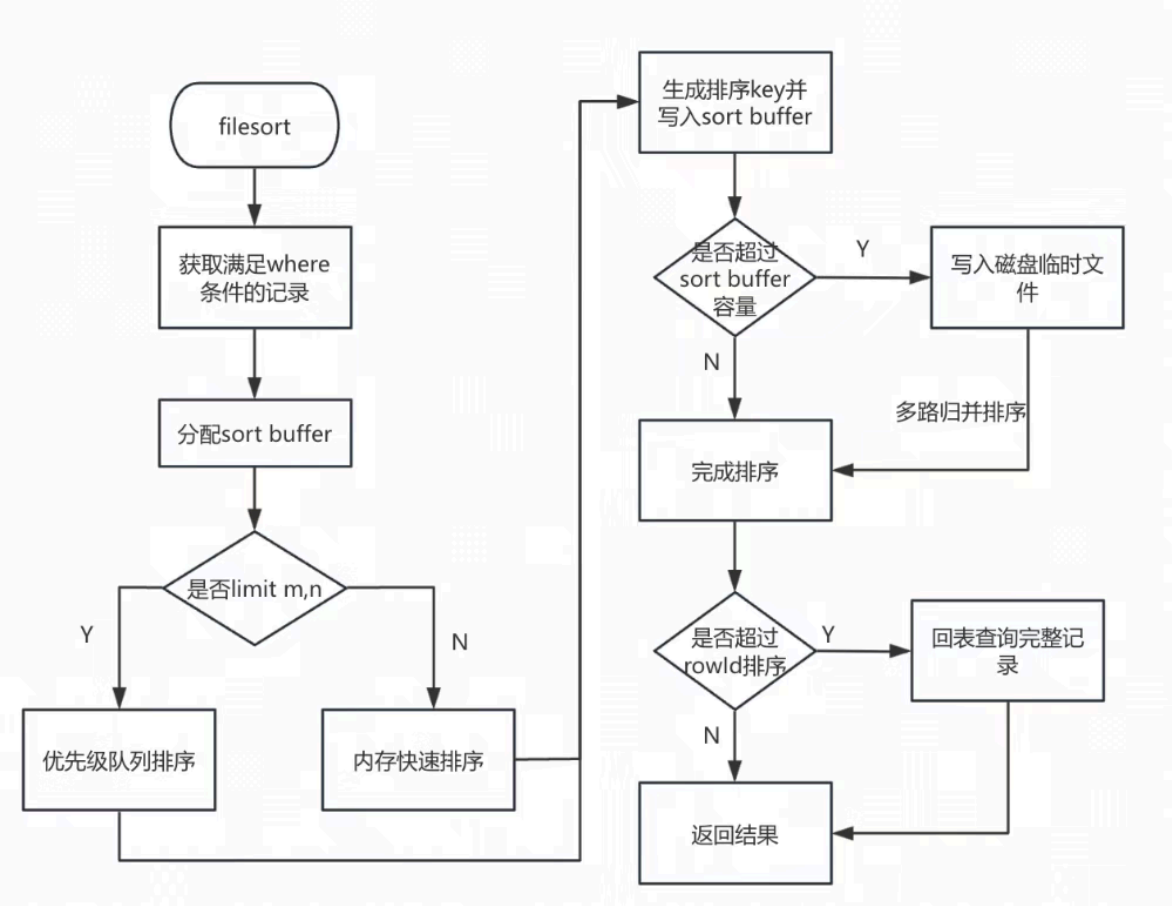
```
select * from tcorder order by parent_id;
select * from tcorder order by parent_id,from_group;
select * from tcorder where parent_id=1234 order by from_group;
select * from tcorder where parent_id>1234 order by parent_id;
select * from tcorder where parent_id=1234 and from_group>1 order by fr
select * from tcorder order by parent_id desc,from_group desc;
```

如下情形不能走索引排序：

```
select * from tcorder order by from_group;
select * from tcorder order by abs(parent_id);
select * from tcorder where parent_id>1234 order by from_group;
select * from tcorder where from_group=1 order by parent_id;
select * from tcorder order by parent_id desc,from_group asc;
```

不能使用索引排序时，explain就会显示Using filesort，这里的filesort是一个统称，如果排序的数据量小则在内存通过快速排序算法排序，这块内存叫sort buffer，通过sort_buffer_size控制大小，默认为2m，基于DB链接维度分配；如果数据量大则借助磁盘临时文件通过多路归并算法排序，因为磁盘的读写效率远低于内存，所以要避免使用磁盘临时文件排序。另外如果order by后面跟着limit m,n的话，则会优化成基于内存的优先级队列的方式排序，但是如果m+n条数据超过了排序内存的容量还是会退化到借助磁盘临时文件排序的方式，所以要避免非必要的深翻查询，容易导致慢SQL。

两种排序方式都是基于排序key来做排序，排序key前面是参与排序的字段值，如果有字符串类的字段参与排序，则最大只允许max_sort_length字节（默认是1024）的数据参与排序。排序key后面的部分默认是其他的需要返回的字段，称为全字段排序；如果排序key的总长度超过max_length_for_sort_data（默认是1024），则排序key后面的部分变成主键ID，称为rowId排序，从而减少排序key的长度，避免使用磁盘临时文件排序，等整体排序完成后再根据这个主键ID回表获取需要返回的各个字段。filesort整体的流程如下：

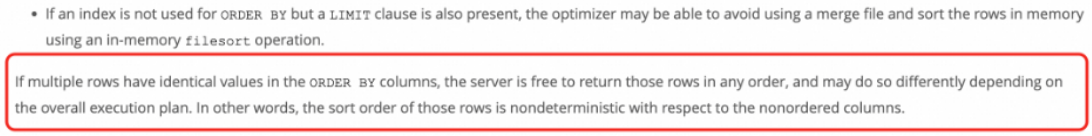


实际的排序过程中是否触发了优先级排序或者临时文件排序，可以通过optimizer_trace日志来做确认，该日志主要用来跟踪优化器的执行过程，默认关闭，通过set optimizer_trace="enabled=on";开启，另外为了保证trace日志能被完整保留，需要调整缓存大小set optimizer_trace_max_mem_size=1638400；注意这两个变量修改只能DBA黑屏操作。如下：

```
{
            "filesort_information": [--排序的字段和顺序
              {
                "direction": "desc",
                "table": "`tcorder`",
                "field": "create_time"
              },
              {
                "direction": "asc",
                "table": "`tcorder`",
                "field": "order_id"
              }
            ],
            "filesort_priority_queue_optimization": {
              "limit": 10,--查询SQL中带了limit 10
              "rows_estimate": 285372816,--预估的参与排序的总行数
              "row_size": 67,--预估的排序key的大小
```

```
            "memory_available": 2097152,--sort buffer的大小
            "chosen": true--说明使用了优先级队列排序
          },
          "filesort_execution": [
          ],
          "filesort_summary": {
            "rows": 11,--最终返回给上层的行数
            "examined_rows": 10597,--实际处理的行数
            "number_of_tmp_files": 0,--使用的临时文件的数量，为0表示未生成
            "sort_buffer_size": 832,--实际使用的sort buffer的大小
            "sort_mode": "<sort_key, additional_fields>"--排序key的构
          }
        }
```

注意mysql排序使用的三种算法快速排序、优先级队列排序和多路归并排序 都存在键值相同的记录排序顺序不稳定的问题，这并不是mysql的bug而是SQL规范并不要求在 ORDER BY 键值相等时保持输入顺序不变，mysql的文档也明确说明了该问题：



- If an index is not used for ORDER BY but a LIMIT clause is also present, the optimizer may be able to avoid using a merge file and sort the rows in memory using an in-memory filesort operation.

If multiple rows have identical values in the ORDER BY columns, the server is free to return those rows in any order, and may do so differently depending on the overall execution plan. In other words, the sort order of those rows is nondeterministic with respect to the nonordered columns.

为了保证排序结果的稳定，只能基于索引排序或者把主键ID加入到排序字段中。



分析慢SQL类型和根因

1. 基于dbTrace的离线日志分析捞取最近一周慢SQL量级前20的慢SQL的SQL Parttern ID和主要的调用来源应用。
2. 根据SQL Parttern ID和调用来源应用名到dbTrace在线日志中查询最新的慢SQL日志，从日志中拿到带有参数的慢SQL和最新的traceId。
3. 通过explain命令分析带有参数的慢SQL，同时根据SQL Parttern ID去dbservice的慢SQL监控中查询该慢SQL的执行统计信息初步确认慢SQL原因。
4. 第三步排查下来发现并非索引问题，需要进一步排查产生慢SQL时DB的CPU/活跃链接数/磁盘读写/网络带宽等以及应用机器的GC/网络丢包等是否有异常，如果存在异常需要先排查异常的根因。

5. 拿着最新的traceId找来源应用的owner了解上游业务场景，跟业务沟通确认能否调整查询条件或者切换查询接口，优先考虑业务调整的方式来解决慢SQL。
6. 明确只能通过索引变更的方式来解决的慢SQL的类型和量级，这些也是索引变更完成后用来评价新索引是否符合优化预期的重要依据。

分析全量查询SQL构成

1. 从dbTrace离线表捞取待优化表的最近一周排名前100的SQL Parttern ID、带问号的SQL文本和对应的量级的占比。
2. 从SQL文本中解析出来查询字段，结合对应SQL的查询量级算出来不同字段的查询量级和占比。
3. 明确各个查询字段的具体含义、在实际业务中的用法，据此推算出该字段的区分度。

确认索引字段顺序

联系DBA、SRE、应用owner、leader一起讨论，结合前面2步的分析结果确认二级索引中多个字段的顺序，参考以下原则：

1. 分库分表的字段排在第一位，比如buyer_id/seller_id，这个字段是区分度最大的。
2. 参与排序的字段排在第二位，比如create_time，避免通过filesort排序。
3. 查询频率高且区分度高且的字段排在中间。
4. 查询频率高但区分度低的字段排在最后。
5. 查询频率低的字段参考实际的慢SQL情况，可以考虑不放在索引中，比如gmt_modified，可以引导上游业务切换查询字段。如果慢SQL比较多且上游强依赖的，则将其放在最后。

提交结构设计工单

索引字段顺序确认完成后就需要在dms上提结构设计工单来完成索引变更，先在日常库执行，走完审批流后再在生产库上执行。注意一下几点：

1. 日常库执行只能验证索引变更的SQL本身是否正确，无法直接在日常库验证新索引的效果，比如日常库加完索引后执行explain发现还是走老索引，此时并不能说明新索引比老索引差，至少日常库的数据量级和特征分布无法跟线上库对齐，需要以线上库的验证结果为准。
2. DRC同步时不会过滤alter table这类语句，因此存在多集群的情况下，只需要对中心集群的逻辑库提变更工单即可。另外，因为alter table语句也是binlog日志的一部分且一个DB实例只有一个有效binlog文件，DRC同步在其他集群执行该DDL语句时以及备库同步该DDL语句时都会阻塞掉其他正常的数据变更，导致单元间DRC同步延迟，执行变更期间需

要监控DRC同步延迟是否超出预期，如果同步延迟大幅上涨，需要联系DBA调大执行索引变更各分表间的时间间隔，比如从10s调整到1m，DRC可以利用这1m来追平延迟的变更。延迟过高时一旦主库挂掉，发生主备切换，因为备库需要先回放未同步完的binlog日志，导致备库有较长的时间不可写入。

3. 当前RDS支持异步DDL，即执行DDL语句时不用等待其执行完成即可返回，但是当前只支持用于磁盘碎片整理的optimize table语句生效，且只支持对备库生效。
4. 线上执行时需要选择起始执行时间，所有DDL类变更必须在凌晨业务低峰期执行，第二天早上7点前停止，从而降低出现问题时的影响面。结构设计工单只能指定执行的起始时间，无法设置任务停止时间，只能人工暂停或者依赖DB维度的变更窗口期暂停，一般核心DB会配置变更窗口期，建议联系DBA确认。
5. 压测验证都是针对影子表，需要同时给线上表和影子表创建结构设计工单，避免遗漏导致压测失真。

线上灰度

1. 结构设计工单线上执行的时候支持按照分表、分库和实例三个维度来做灰度，如果实例数较少建议选择分库维度，否则选择实例维度，原则是第一批灰度1%左右的查询流量。灰度行为支持继续自动执行后续任务和暂停任务等待人工干预两种，自动执行风险不可控，必须选择第二种。注意第一次执行时是由dms平台随机选择一个分库或者实例来执行，无法指定，等第一次执行完成后才能由工单执行人来手动选择执行的分库或者实例。
2. dms按照分库或者实例维度来执行DDL工单时，是按照分库加分表维度串行执行，一个实例只能有一个分表执行变更，先变更完某个分库下所有分表才会变更下一个分库，可以在工单执行详情中查看已经变更完成的和正在变更的分表。这样设计会拉长整体的灰度观察时间，大幅降低变更风险，方便基于分库或者分表维度来排查问题，在变更期间需要密切关注监控大盘是否有异常抖动、星环日志画像是否有新增的异常日志。
3. 第一批分库或者实例维度的索引变更完成且确认没有异常后 需要联系DBA 执行老索引的静默，老索引静默后会正常更新但是对外不可见。一旦新索引有问题，先取消老索引的静默并将新索引静默掉，确认新索引问题后再提工单删除新索引。注意索引静默需要特殊的SQL语句，dms不支持，只能DBA黑屏操作。
4. 索引静默完成后，需要挑选一条真实的慢SQL在dms上执行explain命令，验证新索引生效了，老索引不可见，验证没问题后，需要线上观察一周左右。
5. 第一批索引变更线上观察期间可以基于dbTrace日志配置指定分库指定SQL Parttern Id的RT监控以及慢SQL量级监控，通过对比索引变更完成的分库和未变更的分库两者之前的RT、慢SQL量级的差异，注意选择对比的分库间慢SQL的量级需要接近，避免慢SQL差异把整体的平均RT拉长。因为不同分库间数据分布有轻微的差异，在交易场景比较典型的就是大买家的分布，会导致不同分库的查询RT天然就会不同，直接做比较没有太大意义，只

要两者的差异不大即可，重点关注慢SQL量级的变化，同一个分库变更前后一天内的慢SQL量级可以根据dbTrace日志轻松获取。

压测验证

1. 第一批索引变更完成，观察3天左右确认RT没问题且慢SQL量级有显著下降后需要执行对应的影子表的索引变更工单，同样按照分库或者实例维度灰度，第一批索引变更完成后同样需要执行老索引的静默。
2. 准备压测模型，注意压测模型必须要覆盖本次索引变更需要解决的慢SQL对应的业务场景，压测量级根据去年双11的量级计算出单实例压测的量级。
3. 同时压测变更完成的实例和未变更的实例，对比两者间的cpu、活跃链接数、逻辑读，读写RT的差异，持续施压时观察两者的差异是否变大。
4. 持续施压时需要检查压测用户的数据分布是否符合预期，比如订单场景需要校验单个用户下总的订单量，订单量较小时新老索引的差异不会明显，因为数据量小时数据基本都在Buffer Pool中，即使全表扫描也很快。

全量验证

1. 线上观察一周确认RT没问题、慢SQL量级有显著下降且压测验证没问题后可以执行第二批分库或者实例的索引变更，优先执行第一批用于对比的分库，这样可以对比同一个分库变更前后的平均RT变化，规避数据分布差异的影响。第二批灰度5%，第三批灰度20%，第四批剩余所有的，每批优先灰度慢SQL较多的分库并用同样的方式观察一周左右，整体耗时1个月左右完成线上变更。
2. 线上变更完成后再一次性完成影子表的索引变更工单以及老索引静默。
3. 双11压测和大促验证通过后再提工单正式删除静默掉的老索引。



这里说的慢SQL是应用侧感知的慢SQL，不是DB内核统计的慢SQL，DB执行慢、网络抖动或者应用GC都会导致慢SQL。另外慢SQL整体可以分为两类，查询慢SQL和更新慢SQL，更新慢SQL通常是行锁等待导致的，即存在多个大事务同时更新一行记录，其他事务更新时必须等已经抢占到行锁的事务提交完成后才能抢占到行锁，可以借助dbTrace日志快速排查出现并发冲突的两个大事务，通常是调整业务逻辑或者加上全局限流来规避大事务的并发。

查询类慢SQL的主要特征就是处理过程中平均逻辑读和同步物理读比较多，解决慢SQL的核心就是充分利用索引的二分查找特性来降低逻辑读，常见的问题原因和治理措施如下：

问题原因	说明	治理措施
区分度大的字段上没有索引，此时需要回表查，遍历大量记录来找到满足指定条件的记录，产生大量随机IO，性能甚至比顺序IO的全表扫描要慢。	区分度差的字段（比如状态类字段）没有索引时也会回表查，但是因为过滤性不强，只需遍历少量记录即可找到满足条件的记录，不会导致慢SQL	将过滤性强的字段
二级索引下，范围查询的字段排在前面，等值查询的字段排在后面	根据上述二级索引的数据结构可知，等值查询的字段排在后面无法利用索引过滤，排在前面时可以走索引过滤，进一步缩小范围查询遍历的索引记录数，即降低逻辑读	将等值查询的条件排在范围查询的前面
like查询通配符%在前面，此时相当于是判断某个字段是否以指定字符串结尾，无法利用索引来做过滤	字符串类的字段在索引中的顺序是基于字符串本身的字符，从左到右依次比较的，所以判断某个字段是否以指定字符串开头，即like查询通配符%在后面，可以利用索引的有序性来缩小索引遍历范围	更改查询方式
索引字段的查询条件带有函数表达式，此时无法在SQL优化阶段确认函数的值，依赖运行时索引字段的实际值来做函数计算，所以无法利用索引	and ifnull(create_time, 0) = SUBDATE(now(), INTERVAL 51 DAY) ifnull的结果无法提前确认，无法利用索引 and create_time > SUBDATE(now(), INTERVAL 51 DAY) SUBDATE的值可以提前计算，可以利用索引	更改查询方式

优化器索引选择错误	MySQL优化器依赖的统计信息来源于Innodb的周期性索引页采样机制，没有动态采样或者执行反馈能力的，即优化器无法感知实际的数据分布，如果存在多个可用索引时可能导致索引选择错误	合理设计索引，较少可用的索引个数
fillsort排序	filesort排序需要把满足条件的记录都查询出来才能执行排序动作，导致执行过程中产生的逻辑读远远大于预期；另外filesort排序的记录数过多会产生磁盘临时文件，进一步导致RT变长	调整为基于索引排序
查询的数据在索引B+Tree的左侧但是按照倒序排序	利用索引来排序时会根据排序的方向来调整索引叶子节点的读取方向，比如倒序排，则从右往左遍历，正序排则从左往由遍历。如果查询的数据在索引B+Tree的左侧，但是按照倒序排序则意味着需要从右到左遍历大部分的索引记录，增加没有意义的逻辑读。	根据实际的业务场景，调整排序顺序。比如订单列表场景，通常是查询最近1周内的数据，此时应该使用create_time desc，不能使用create_time asc。
count查询命中的记录数过大	count查询执行时会把满足条件的所有记录都捞出来，导致大量的逻辑读，容易产生慢SQL。mysql下无法直接优化，只能结合具体的业务场景考虑使用count limit子查询，mysql执行时判断count的结果达到了limit的限制后会中断扫描直接返回结果，从而减少逻辑读。	结合上游的业务场景分析去掉非必要的count查询或者使用count limit子查询，大多数场景下不需要一个精确的count数字的，比如某个用户最近1个月内退款超过100笔时就不允许参加指定优惠活动，此时不需要count所有的退款记录，只需要count limit 101笔即可。
应用GC	GC时会STW，导致应用侧感知到的RT较长，此时通常	应用GC治理

	是1s内产生多个慢SQL，可以根据dbTrace的慢SQL日志快速确认	
活跃链接数高，cpu不高	通常是磁盘读写达到瓶颈，IO处理变慢导致活跃链接数变高	磁盘升配
CPU高，通常此时活跃链接也会高	慢SQL较多时会产生大量的逻辑读，CPU处理不过来，导致其他正常的查询请求的CPU等待时间变长，活跃链接数升高	增加限流保护，推动慢SQL治理
主备切换	备库变主库时，buffer pool缓存的数据较少，产生大量的磁盘读，容易产生慢SQL	无
网络丢包	DB对外的网络带宽出现尖刺时，受网络丢包影响可能导致应用侧的RT边长，产生慢SQL	排查产生带宽尖刺的原因，通常是同一类列表查询请求被定时触发，并发过高导致，调低并发值即可



- 13.7.5.30 SHOW PROFILE Statement：https://dev.mysql.com/doc/refman/5.7/en/show-profile.html
- 8.2.1.14 ORDER BY Optimization：https://dev.mysql.com/doc/refman/5.7/en/order-by-optimization.html
- 8.2.1.17 LIMIT Query Optimization：https://dev.mysql.com/doc/refman/5.7/en/limit-optimization.html
- MySQL Order By排序的底层原理以及filesort排序：https://juejin.cn/post/7132125495633641508
- MySQL · 引擎特性 · 排序实现：http://mysql.taobao.org/monthly/2020/03/09/

团队介绍

本文作者业空，来自淘天集团–营销&交易技术团队。本团队承担淘天电商全链路营销交易技术攻坚，致力于通过技术创新推动业务增长与用户体验升级。过去一年主导了多个高价值项目，包括：支撑618、双11、春晚等亿级流量洪峰、构建业界领先的全网价格力体系、承接淘宝全面接入微信支付、搭建集团最大的AI创新平台–ideaLAB，支撑淘宝秒杀等创新业务的高速增长。

营销&交易技术是淘天集团核心技术团队，团队年轻充满活力，重视新人成长，技术氛围浓厚，鼓励突破性思考与实践。 期待你是：

- 代码极客：编程能力强悍，痴迷代码与设计之美。
- AI 先锋：关注AIGC、大模型等技术趋势，用技术思维撕裂业务天花板，敢想敢造；
- 创新行动派：好奇心强，乐于接受挑战，用技术思维突破业务边界。

岗位范围：java后端、前端、后训练、评测、agent 开发、测开等，欢迎加入我们，一起快乐工作、认真生活 🏝 。 简历可投递邮箱：chenglin.ycl@alibaba–inc.com。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 1.0万

留言 3

写留言



宋蛋 北京 2025年11月19日 13

在AI时代看到一篇工程文章太不容易了



懒懒 浙江 2025年11月20日

哇哦，厉害了👍



孙永贺 浙江 2025年11月19日

满满的干货



张雪教育 广告

在职研究生：学费7980元，2年制免到校，学信网可查，国家认可🤌



进入小程序